

Steam games and player counts

Group 9 | Data Engineering Project 1

Objective: Analyze the popularity and player activity of Steam games and investigate how game characteristics such as genre, price, release date and developer relate to player counts over time.

Stakeholders: Game developers, game publishers, product managers and market analysts who want to understand player engagement and market trends on Steam.

KPIs: Average player count; Peak concurrent player count; Monthly growth / decline (%).

Business questions

1. Which steam games have the highest average monthly player counts?
2. Which primary genres have the highest average activity per month?
3. How does player activity change over time?
4. Is current game price associated with historical player popularity?
5. Which games have the highest average-to-peak concurrent player ratio over time?

Datasets

Dataset 1 - Steam game metadata (SteamSpy API and/or Steam Store API). One checked SteamSpy API page contains 1000 records and 17 fields. **Possible attributes:** appid, name, developer, publisher, genre, release_date, price, positive_reviews, negative_reviews, owners, average_playtime.

Dataset 2 - Historical Steam player data

(<https://www.kaggle.com/datasets/jackogozaly/steam-player-data>). The CSV contains 5271 rows and 8 columns, covering monthly player activity for 98 games from July 2012 to September 2021. **Possible attributes:** Month_Year, Avg_players, Gain, Percent_Gain, Peak_Players, URL, Date, Game_Name.

Integration and refresh: The Steam appid is extracted from each game's URL and used to join the player data with Dataset 1. Historical data will initially be loaded for modelling and analysis. In the proposed production architecture, current player counts would be collected hourly from the Steam Web API and aggregated into monthly averages and sampled peaks.

Tooling

Airflow is used for ingestion orchestration; MongoDB for raw storage; Python for loading PostgreSQL staging; dbt for cleaning, transformation and data quality testing; PostgreSQL hosts the star schema; Superset provides reporting; Docker provides containers.

Group Roles & Contributions

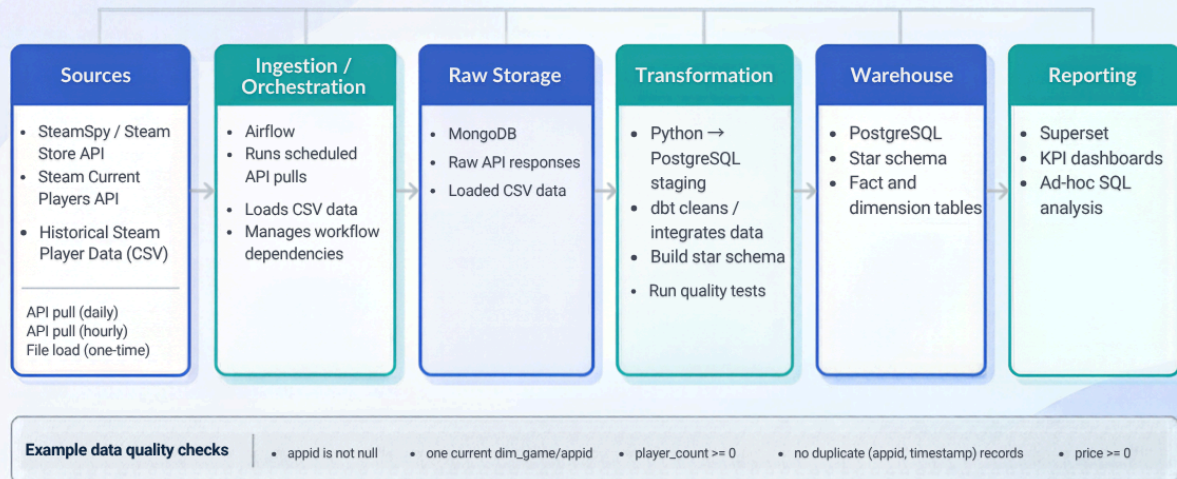
Steven Laul - business brief, datasets and data architecture (34%); Lauri Löösi - tooling, data model and SCD justification (33%); Richard Prost - data dictionary, SQL queries and GitHub documentation (33%).

LLM disclosure & Github

ChatGPT was used for writing improvements and in the end squeezing an 8 page summary into 3 pages;) Canva AI to make graphs visually more appealing.

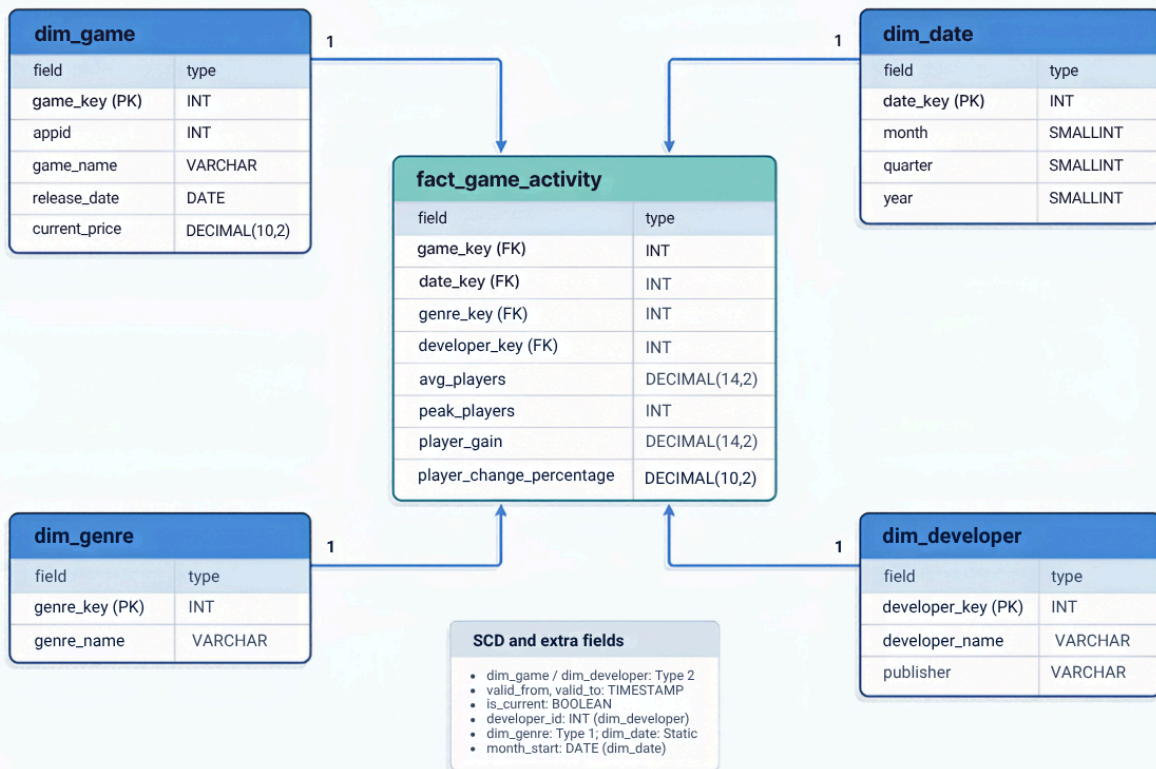
GitHub repo: <https://github.com/rprost/DataEngineeringProject>

Steam Game Analytics – Data Architecture



Steam Game Analytics Star Schema

Grain: one row = one Steam game in one month



SCD choices and mapping rules

dim_game uses **Type 2** for changes to name, release date or price; **dim_developer** uses **Type 2** for name/publisher changes under a curated developer_id. Both use valid_from, valid_to and is_current. **dim_genre** uses **Type 1** for label corrections; **dim_date** is **static**. Use the first listed genre, developer and publisher as primary values. Grain: one appid per month. Historical facts use the initial metadata snapshot; Type 2 history starts when collection begins.

Data dictionary

fact_game_activity - monthly player activity: **game_key**, **date_key**, **genre_key**, **developer_key** INT - foreign keys to the four dimensions; **avg_players** DECIMAL(14,2) - monthly mean concurrent players; **peak_players** INT - monthly peak concurrent players; **player_gain** DECIMAL(14,2) - change in the monthly mean; **player_change_percentage** DECIMAL(10,2) - percentage change from the previous monthly mean.

dim_game - game versions: **game_key** INT - surrogate primary key; **appid** INT - Steam application ID; **game_name** VARCHAR(255) - game title; **release_date** DATE - release date; **current_price** DECIMAL(10,2) - USD price recorded in this version.

dim_date - calendar months: **date_key** INT - primary key in YYYYMM format; **month_start** DATE - first day of the month; **month**, **quarter**, **year** SMALLINT - calendar attributes.

dim_genre - primary genres: **genre_key** INT - surrogate primary key; **genre_name** VARCHAR(100) - genre label.

dim_developer - developer versions: **developer_key** INT - surrogate primary key; **developer_id** INT - curated stable developer/publisher relationship ID; **developer_name**, **publisher** VARCHAR(255) - primary developer and publisher names.

Both Type 2 dimensions: **valid_from** TIMESTAMP - inclusive validity start; **valid_to** TIMESTAMP - exclusive validity end, NULL for the current version; **is_current** BOOLEAN - current-version flag. Validity timestamps use UTC.

Missing player gains and percentage changes remain NULL; percentage change is also NULL when the previous mean is zero. Facts contain **one row per appid/month**.

Demo queries

1. Highest average monthly player counts

```
SELECT g.appid, MAX(g.game_name) game_name,
AVG(f.avg_players) AS avg_players
FROM fact_game_activity f
JOIN dim_game g USING(game_key)
GROUP BY g.appid
ORDER BY avg_players DESC LIMIT 10;
```

2. Primary genre activity by month

```
SELECT d.year, d.month, ge.genre_name,
AVG(f.avg_players) AS avg_players
FROM fact_game_activity f
JOIN dim_date d USING(date_key)
JOIN dim_genre ge USING(genre_key)
GROUP BY d.year, d.month, ge.genre_name
ORDER BY d.year, d.month, avg_players DESC;
```

3. Player activity over time

```
SELECT g.appid, g.game_name, d.year, d.month,
f.avg_players, f.peak_players, f.player_gain,
f.player_change_percentage
FROM fact_game_activity f
JOIN dim_game g USING(game_key)
JOIN dim_date d USING(date_key)
ORDER BY g.appid, d.year, d.month;
```

4. Current price vs historical popularity

```
WITH popularity AS (
SELECT g.appid, AVG(f.avg_players)
avg_players
FROM fact_game_activity f
JOIN dim_game g USING(game_key)
GROUP BY g.appid)
SELECT CORR(
g.current_price::DOUBLE PRECISION,
p.avg_players::DOUBLE PRECISION)
AS price_player_correlation
FROM popularity p
JOIN dim_game g USING(appid)
WHERE g.is_current
AND g.current_price IS NOT NULL;
```

5. Average-to-peak player ratio

```
SELECT g.appid, MAX(g.game_name) game_name,
AVG(f.avg_players/NULLIF(f.peak_players,0))
*100 AS avg_to_peak_pct
FROM fact_game_activity f
JOIN dim_game g USING(game_key)
GROUP BY g.appid
ORDER BY avg_to_peak_pct DESC NULLS LAST;
```